Software Architecture and Design Specification (SAD)

Project: Traffic Management System (Simulation)

Version: 1.1

Authors: Aarya, Vijeth, Vaibhav, Vainik

Date: 11-11-2026

Status: Implementation verified; team approval pending

Approvals

Role	Name	Signature/Date
Course Coordinator		

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Traffic Management System (TMS) Simulation, a C++17 discrete-time traffic simulator.

1.2 Scope

Covers scenario loading, vehicle generation and queues, adaptive signal control, road network and routing, movement, congestion and incident handling, metrics and reporting through a console interface.

1.3 Audience

Developers, testers, reviewers, instructors and maintainers.

1.4 Definitions

TMS, SRS, FR, NFR, RTM, Tick, Lane Queue, Signal Phase, RAII, ADR, STRIDE.

2. Document Overview

2.1 How to use this document

Section 3 describes architecture (components, pattern, traceability, security). Section 4 describes design (sequence diagrams, API, error handling).

2.2 Related Documents

TMS SRS v1.0, TMS Software Test Plan v1.0, project plan in the repository.

3. Architecture

3.1 Goals & Constraints

- Goals: modular, testable, deterministic, safe (no conflicting greens), reproducible.
- Constraints: C++17 and CMake on Linux; in-process modules only; no real hardware; no mutable globals except a controlled simulation context (NFR-01, NFR-05).

3.2 Stakeholders & Concerns

Stakeholder	Concerns
Scenario Designer / Operator	Usability, clear errors, repeatable runs
Traffic Analyst	Correct metrics and reports
Developers / Testers	Modularity, testability, deterministic behaviour
Evaluators	Traceability to SRS, quality, security

3.3 Component (UML) Diagram

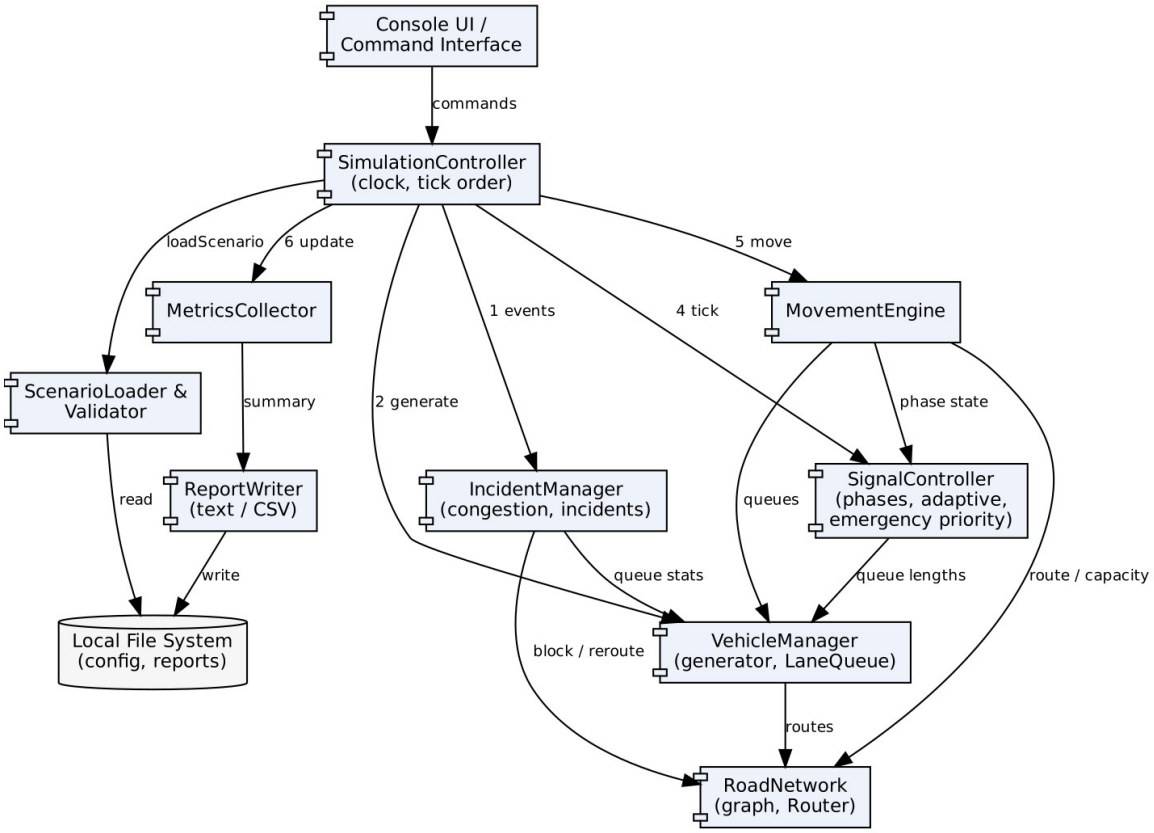


Figure 1: Component diagram. Numbers on SimulationController edges show the fixed tick-processing order.

3.4 Component Descriptions

Component	Responsibility	Key operations
Console main	Batch CLI validation, preview, run, trace and CSV output	--validate; --scenario; --trace; --report
Simulation	Owns clock, modules and deterministic tick order	configureScenario(), step(), run(), reset()
ScenarioLoader	Bounded parsing and atomic setup validation	loadFile(), load(), validate()
VehicleManager and LaneQueue	Seeded IDs, arrivals, FIFO queues and pending retry	generate(), advanceArrivals(), laneSnapshots()
SignalController	Adaptive timing, configured safety and FIFO priority	advance(), requestPriority(), permits()
RoadNetwork	Directed graph and deterministic weighted routing	addRoad(), neighbors(), shortestRoute(), findRoad()
MovementEngine	Capacity-limited travel, safe crossing and emergency requests	advance(), readyMovementQueues(), requestEmergencyPriority()
Incident and congestion modules	Schedule road restrictions and detect thresholds; Simulation invokes suffix responses	schedule(), advance(), evaluate(); respondToTraffic()
MetricsCollector	Completed/unfinished, waiting, queue peak and throughput	recordCompleted(), observeQueue(), snapshot()
ReportWriter	Final CSV; CLI prints readable text and checks file close	writeCsv(stream, report)

3.5 Chosen Architecture Pattern and Rationale

Pattern: Modular, layered architecture with a central orchestrator (SimulationController) and a time-stepped (tick-based) simulation loop. Layers: Presentation (Console UI) → Orchestration (SimulationController) → Domain modules (Vehicle, Signal, Network, Movement, Incident) → Services (Metrics, Report, file I/O).

Rationale: separates concerns, lets each team member own one module, allows each module to be unit-tested through its interface (NFR-01, NFR-02), and keeps a single stable tick order that makes runs deterministic (FR-18). Rejected alternatives: event-driven/actor model (more complex, harder to make deterministic); microservices (unnecessary for a standalone simulator).

ADR-1: Dijkstra uses enabled directed edges weighted by travel time, with lexicographic road-ID ties. ADR-2: VehicleManager owns one seeded std::mt19937_64. ADR-3: Apply incidents; generate/admit; evaluate congestion; respond by suffix rerouting; request priority and advance signals; observe queues; move; count waits/completions and final congestion; increment the clock.

3.6 Technology Stack & Data Stores

Concern	Choice
Language / build	C++17, CMake
Testing	GoogleTest v1.15.2, CTest, GCC/gcov JSON line and branch reports
Quality	clang-format 18.1.8; compiler warnings as errors;

Concern	Choice
	Clang static analyzer. Sanitizers optional and not claimed as run.
CI/CD	GitHub Actions jobs cover tests, formatting, analysis, coverage and container smoke. Jenkins/archive and Docker definitions are supplied; external execution is pending.
Data structures	std::deque (lane queues), adjacency list (graph), std::priority_queue (Dijkstra), std::unordered_map (registries)
Persistence	Text scenario files in config/; text/CSV reports on local file system (no database)

3.7 Risks & Mitigations

Risk	Mitigation
Inconsistent module integration	Documented fixed tick order; integration tests
Non-determinism	Single seeded RNG; ordered containers for iteration
Safety violation (conflicting greens)	Conflict matrix checked inside SignalController before any phase change; assertions in tests
Invalid input crashes the program	Central validation in ScenarioLoader and Console UI (NFR-04, NFR-07)

3.8 Traceability to Requirements

Requirement(s)	Component(s)
FR-01, NFR-04	ScenarioLoader & Validator, SimulationController
FR-02, FR-03, FR-04	VehicleManager (Vehicle, LaneQueue)
FR-05, FR-06, FR-07, FR-08	SignalController
FR-09, FR-12	RoadNetwork (Router)
FR-10, FR-11	MovementEngine
FR-13, FR-14, FR-15	CongestionDetector, IncidentManager, Simulation response and RoadNetwork
FR-16	MetricsCollector
FR-17	ReportWriter
FR-18, NFR-10	VehicleManager (seeded RNG), Simulation reset, report seed
NFR-01, NFR-02, NFR-08	All modules via interfaces; RTM
NFR-03	Simulation loop, MovementEngine, Router
NFR-05, NFR-09	CMake, CI, static analysis
NFR-06	Console UI, ReportWriter
NFR-07, SEC-01 to SEC-04	Console UI and ScenarioLoader validation, SignalController safety check, RAII throughout

3.9 Security Architecture

The system is a local, standalone, single-user simulator, so the main threats come from malformed inputs and misuse of commands. Threat model (STRIDE):

Threat	Example	Control	Req
Spoofing	Not applicable (no authentication, local use)	N/A – documented assumption	–
Tampering	Edited scenario file with impossible values	Schema and range validation; reject on any error	SEC-01, SEC-02
Repudiation	Untraceable run	Log seed and config in every report	NFR-10

Threat	Example	Control	Req
Information disclosure	Path leaks in error messages	Messages state cause only, no internal paths/memory	NFR-06
Denial of service	Huge file or extreme parameters hang the run	Size and value limits; capacity/duration caps	SEC-01, SEC-02
Elevation of privilege / safety bypass	Emergency request forces conflicting greens	Conflict check inside SignalController, cannot be bypassed by callers	SEC-04

Security objectives SO-1 Integrity, SO-2 Availability and SO-3 Safety are defined in SRS Section 6. SEC-01/02 use input and resource bounds; SEC-03 uses RAII, warnings, pinned formatting, static analysis and output-failure checks; SEC-04 requires safe priority and full clearance. Sanitizers are an optional developer check, not claimed as executed CI evidence.

3.10 Division of Work and Module Ownership

Member	Module ownership	Requirements	Interfaces documented
Aarya	Vehicle, LaneQueue, VehicleManager, input validation	FR-01 to FR-04	VehicleManager
Vijeth	SignalPhase, SignalController, signal policy	FR-05 to FR-08	SignalController
Vaibhav	Road, RoadNetwork, Router, MovementEngine	FR-09 to FR-12	RoadNetwork
Vainik	IncidentManager, MetricsCollector, ReportWriter	FR-13 to FR-17	MetricsCollector, ReportWriter
All	SimulationController integration, FR-18, NFRs, CI	FR-18, NFR-01 to NFR-10	Tick-order contract

3.11 Interface Stability and ABI

The application is a statically linked C++17 executable. Public headers under include/traffic are concrete module interfaces, with value objects and a replaceable route-provider callback. There is no shared-library ABI promise. Simulation owns mutable subsystem state; unit tests call modules directly. Changes to public contracts require module-owner review. This concrete-interface design replaces the originally proposed pure abstract interfaces.

4. Design

4.1 Design Overview

Each module is a C++ class (or small group) behind an interface in include/<module>/. The SimulationController calls modules in the fixed tick order. Data is exchanged via validated value objects (Vehicle, Road, Phase), not shared globals.

4.2 UML Sequence Diagrams

Figure 2 – Sequence 1 – Normal tick: vehicle generation, signalling and movement (FR-02 to FR-06, FR-10, FR-11, FR-16).

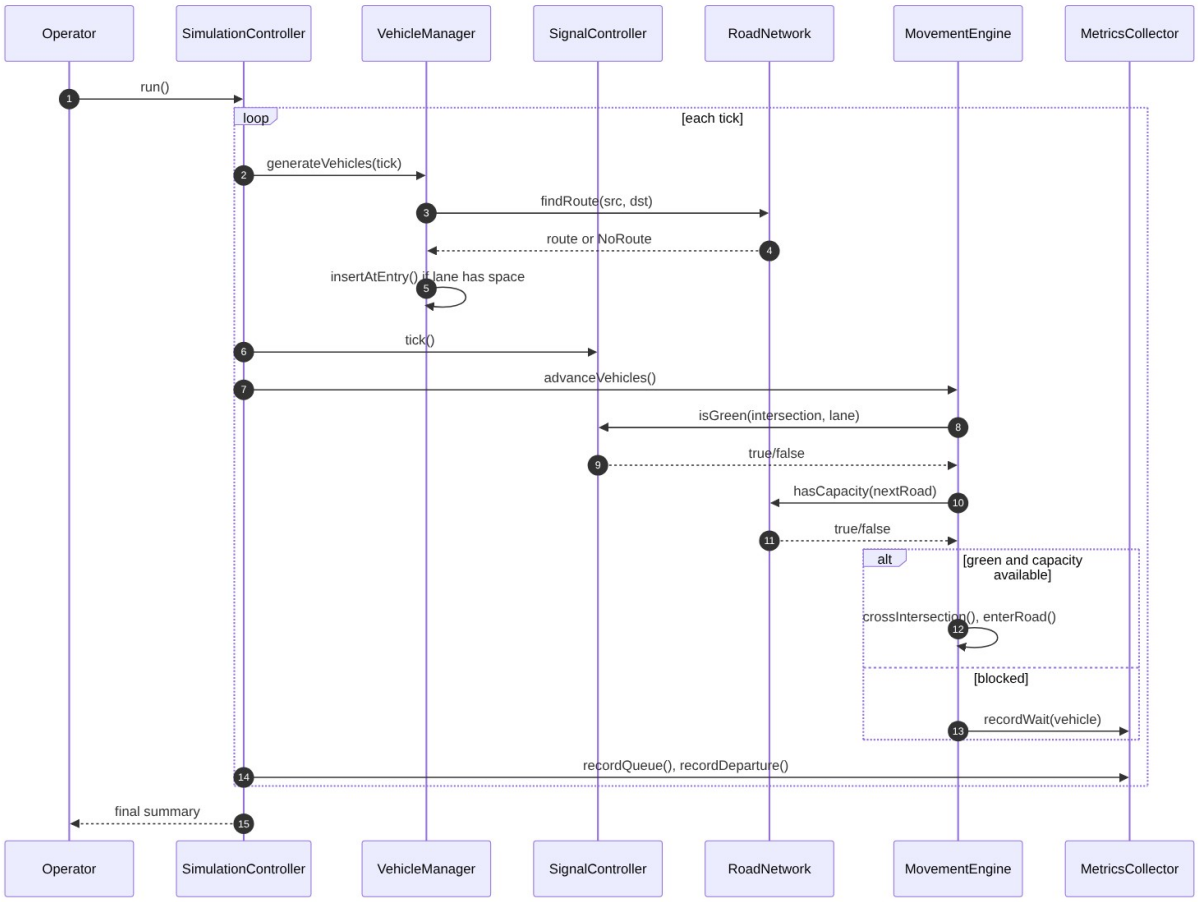
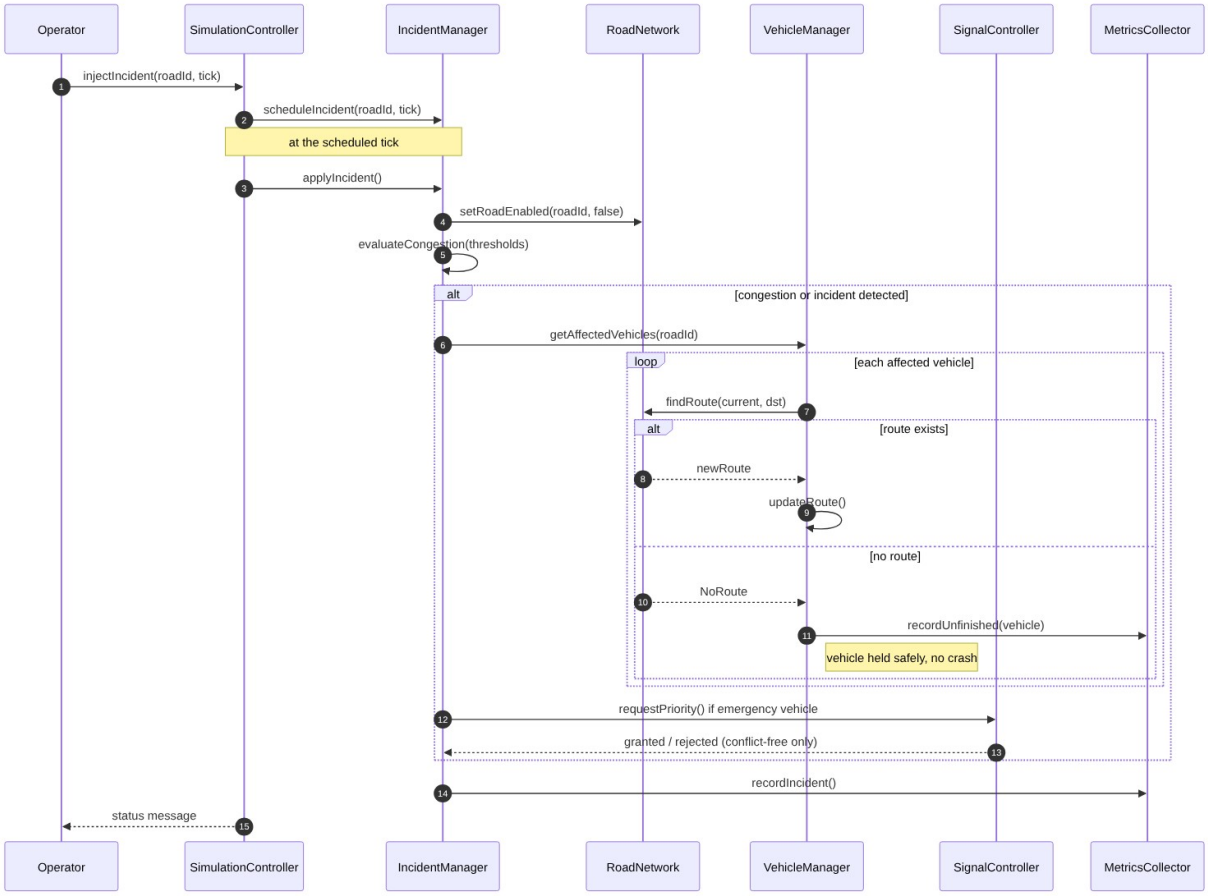


Figure 3 – Sequence 2 – Incident injection, congestion detection and rerouting (FR-12 to FR-15).


Operator
SimulationController
IncidentManager
RoadNetwork
VehicleManager
SignalController
MetricsCollector
1 injectIncident(roadId, tick)
2 scheduleIncident(roadId, tick)
at the scheduled tick
3 applyIncident()
4 setRoadEnabled(roadId, false)
5 evaluateCongestion(thresholds)
alt
[congestion or incident detected]
6 getAffectedVehicles(roadId)
loop
[each affected vehicle]
findRoute(current, dst) 7
alt
[route exists]
8 newRoute
9 updateRoute()
[no route]
10 NoRoute
11 recordUnfinished(vehicle)
vehicle held safely, no crash
12 requestPriority() if emergency vehicle
granted / rejected (conflict-free only) 13
14 recordIncident()
status message 15
Operator
SimulationController
IncidentManager
RoadNetwork
VehicleManager
SignalController
MetricsCollector

4.3 API Design

All communication is in-process through public C++ headers. The following table lists actual module contracts; respondToTraffic is an internal Simulation method.

Interface	Method	Inputs	Returns / Errors
RoadNetwork	shortestRoute(from,to,excluded)	IDs and optional excluded roads	vector<RoadId>; empty if no enabled route
RoadNetwork	findRoad(id), roads()	Road ID	Road pointer or nullptr; read-only map accessor
SignalController	advance(now,queues), permits(...)	Tick and movement counts	Safe timed state; bool permission
SignalController	requestPriority(intersection, movement)	Configured movement	bool; FIFO target phase after clearance
VehicleManager	generate(tick), advanceArrivals(...)	Tick and route callbacks	Generated/admitted/ unroutable IDs
VehicleManager	laneSnapshots(), find(id)	Lane/vehicle IDs	Value snapshots or optional lookup
Simulation	step(), run(), reset()	Validated scenario	One tick, duration run, deterministic reset
Simulation	respondToTraffic()	Incident and congestion state	Preserve current road; replace valid suffix or wait
MetricsCollector	snapshot(unfinished,ticks)	Counts and processed interval	Completed-only average wait and throughput
ReportWriter	writeCsv(stream,report)	Final report value	CSV; CLI checks file close status

Actual console interface: --help; --validate <file>; --scenario <file> [--show-scenario] [--trace] [--lane-status] [--report <output.csv>]. Incidents are scheduled in files, emergency vehicles request safe priority automatically, and Simulation::requestPriority supports in-process requests. No interactive command menu is promised by this revision.

4.4 Error Handling, Logging & Monitoring

- Setup/configuration and operational file failures throw standard exceptions caught at the CLI boundary. Normal no-route outcomes use VehicleState::Unroutable and RouteFailure; queue insertion and priority requests return bool. Rejected routes do not enter queues. Admission-provider exceptions roll back the admission tick.
- Actual CLI errors use [ERROR][Scenario] or [ERROR][Console], readable detail and nonzero exit codes. Numeric error IDs and a separate log-level framework are not implemented; these are documented design deviations.
- Invalid configuration: reject, leave state unchanged, report cause, exit code non-zero for batch runs (NFR-04).
- No-route and full-lane situations are normal outcomes, not crashes (FR-11, FR-15).
- Console output provides seed, optional scenario/tick trace, readable metrics and errors. No separate INFO/WARN logger or optional log-file framework is implemented.

- Monitoring: per-run counters (vehicles generated, completed, unfinished, max queue, rejected inputs) in the final report.

Actual outcome	**Meaning**
0	CLI success
1	Scenario or output failure
2	Invalid CLI usage
false / empty	Normal rejected priority/queue request or missing lookup/route
RouteFailure	NoRoute or InvalidRoute recorded for the individual vehicle

4.5 UX Design

Console menu with numbered options, consistent labels, concise error messages and a readable summary table (NFR-06).

4.6 Module Design: Road Network, Routing and Movement

This module (FR-09 to FR-12) is the core of vehicle progression across the network.

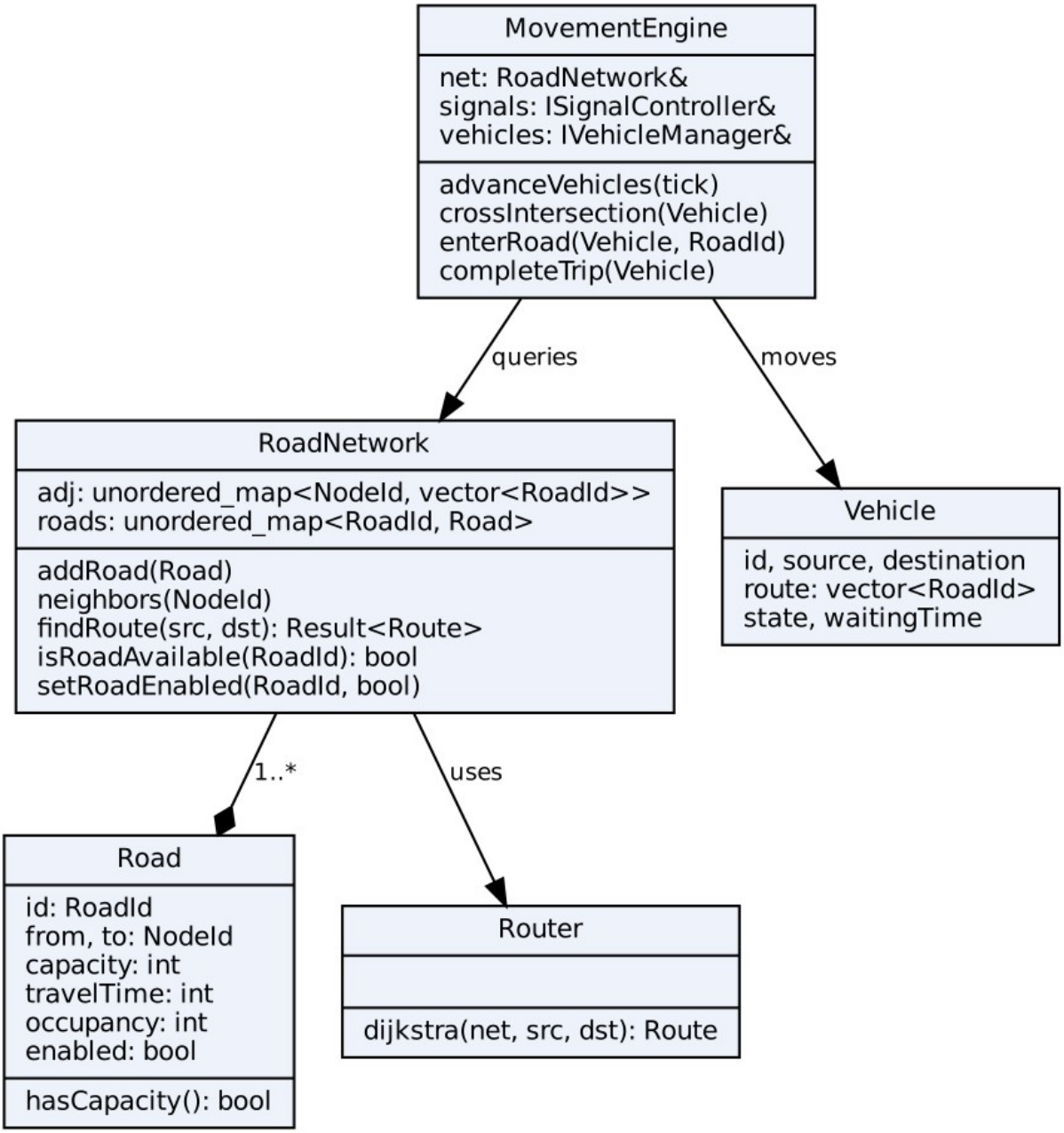


Figure 4: Class diagram of the Road Network and Movement module.

- Graph: adjacency list (unordered_map<NodeId, vector<RoadId>>), roads stored by id. Sparse graph, O(1) neighbour lookup (FR-09).
- Routing: Dijkstra with std::priority_queue; edge cost = travelTime (weighted-path policy). Disabled roads are skipped, so blocked roads are avoided automatically. Complexity O((V+E) log V). Unreachable destination returns NoRoute (FR-12, FR-15).
- Movement rule per tick, evaluated in deterministic order (ascending intersection id, then lane id): a vehicle at the front of its lane moves only if (a) its signal is green, (b) the next road on its route is enabled, and (c) the next road occupancy < capacity. Otherwise it waits and its waiting time increases (FR-10, FR-11).

- Trip completion: when a vehicle reaches its destination node it is removed, its trip time is recorded and the completed counter increments.
- Invariants: occupancy <= capacity for every road and lane at the end of every tick; a vehicle is in exactly one place (lane queue, road or completed).

4.7 Open Issues & Next Steps

- Implemented: versioned conflict records and phase validation; the scenario owner must supply a complete conflict table.
- Remaining acceptance: run external pipelines, record unfamiliar-user review and obtain final owner approval; see submission-checklist.md.
- Implemented: scenario preview, lane snapshots and a readable terminal tick trace. A graphical interface remains optional.

5. Appendices

5.1 Glossary

See SRS Definitions and Acronyms.

5.2 References

IEEE 42010, IEEE 1016, OWASP Secure Coding Practices, NIST SP 800-160.

5.3 Tools

Graphviz, Mermaid, draw.io, GoogleTest, CMake, GitHub Actions.

6 Final implementation contracts

See architecture.md and deviations.md for the current tick order, entry-aware routing, incident/congestion response, safe priority, lane occupancy and metric definitions. The original component and sequence diagrams are retained as conceptual views; concrete calls and the additional response/priority steps are specified by this appendix and the implementation headers.

FR-11 tracks in-transit downstream lane occupancy separately from source waiting queues. Initial entry does not consume downstream-lane occupancy. Reconfiguration and reset replace the entire movement state. Road capacity remains an aggregate independent limit.

The no-alternative response retains a valid existing route and waits; newly generated no-route vehicles become Unroutable. Congestion exclusion falls back to an enabled congested path when no uncongested alternative exists. Seeded replay includes the resulting route changes and CSV metrics.

Measured reports and source/test traceability are in docs/reports and docs/traceability.md. Docker daemon and Jenkins server execution, human usability review, owner approval, Project board verification and sprint freeze require external/team evidence. No signatures or past sprint activity are inferred from automated tests.